

PROGRAMAÇÃO DE COMPUTADORES

Estruturas Condicionais e de Repetição

Prof. Dr. José Jairo de Santana e Silva

Aula 5

Estruturas Condicionais e de Repetição

AGENDA DA AULA

☰ Agenda

Estruturas Condicionais

- Conceito de tomada de decisão
- Condicional simples: `if`
- Condicional composta: `if-else`
- Condicional múltipla: `if-elif-else`
- Condicionais aninhadas
- Operador ternário
- Boas práticas

☰ Agenda

Estruturas de Repetição

- Conceito de loops e iteração
- Loop `while`
- Loop `for` com `range()`
- `break`, `continue` e `pass`
- Loops aninhados
- Contadores e acumuladores
- Exercícios práticos

OBJETIVOS DE APRENDIZAGEM

🎯 Objetivos

Ao final desta aula, o estudante será capaz de:

- Utilizar estruturas condicionais (**if**, **elif**, **else**) para controlar o fluxo de execução
- Implementar laços de repetição (**while** e **for**) de forma eficiente
- Aplicar comandos de controle de fluxo: **break**, **continue** e **pass**
- Combinar condicionais e loops para resolver problemas do mundo real
- Identificar quando usar **for** vs **while**
- Evitar erros comuns como loops infinitos

ESTRUTURAS CONDICIONAIS

❓ O que são Estruturas Condicionais?

Estruturas condicionais permitem que o programa **tome decisões** com base em condições.

O programa avalia uma **expressão booleana** (verdadeira ou falsa) e executa diferentes blocos de código conforme o resultado.

Analogia do dia a dia:

▮ "Se estiver chovendo, leve guarda-chuva. Senão, leve óculos de sol."

Portugol vs Python:

Portugol	Python
se ... entao	if ...:
senao	else:
senao se	elif ...:

</> Condicional Simples: **if**

A forma mais básica de decisão: executa um bloco **somente se** a condição for verdadeira.

Sintaxe:

```
if condicao:  
    # bloco executado se condicao for True
```

Importante: A **indentação** (4 espaços) define o bloco de código em Python. Tudo que estiver indentado após o **if** faz parte do bloco condicional.

</> if - Exemplo: Número positivo

Código:

```
numero = int(input("Digite um número: "))  
  
if numero > 0:  
    print(f"O número {numero} é positivo!")  
  
print("Fim do programa.")
```

Saída (supondo entrada = 7):

```
## O número 7 é positivo!
```

```
## Fim do programa.
```

</> if - Exemplo: Verificar maioria

Código:

```
idade = int(input("Qual sua idade? "))  
  
if idade >= 18:  
    print("Você é maior de idade.")  
    print("Pode tirar carteira de motorista!")
```

Saída (supondo entrada = 20):

```
## Você é maior de idade.  
## Pode tirar carteira de motorista!
```

Condicional Composta: **if-else**

Quando precisamos de **dois caminhos**: um se a condição for verdadeira e outro se for falsa.

Sintaxe:

```
if condicao:  
    # bloco se True  
else:  
    # bloco se False
```

🔗 if-else - Exemplo: Par ou Ímpar

Código:

```
numero = int(input("Digite um número: "))  
  
if numero % 2 == 0:  
    print(f"{numero} é PAR")  
else:  
    print(f"{numero} é ÍMPAR")
```

Saída (supondo entrada = 13):

13 é ÍMPAR

🔗 if-else - Exemplo: Aprovação

Exemplo: Verificar aprovação de aluno

Código:

```
nota = float(input("Digite a nota final: "))

if nota >= 7.0:
    print("APROVADO!")
    print(f"Parabéns! Sua nota foi {nota:.1f}")
else:
    print("REPROVADO.")
    print(f"Sua nota foi {nota:.1f}. Precisa de >= 7.0")
```

Saída (supondo nota = 5.5):

```
## REPROVADO.
## Sua nota foi 5.5. Precisa de >= 7.0
```

💡 Desafio Didático: `if-else`

Escreva um programa que leia a idade de uma pessoa e mostre:

- `Pode votar` se a idade for maior ou igual a `16`
- `Ainda não pode votar` caso contrário

Desafio extra:

- Troque o limite para `18` e adapte a mensagem

Condicional Múltipla: **if-elif-else**

Quando existem **mais de dois caminhos** possíveis.

Sintaxe:

```
if condicao1:
    # bloco 1
elif condicao2:
    # bloco 2
elif condicao3:
    # bloco 3
else:
    # bloco padrão
```

O **elif** é uma abreviação de "else if". Podemos ter **quantos elif** forem **necessários**.

🔗 if-elif-else - Exemplo: Conceito por nota

Código:

```
nota = float(input("Digite a nota: "))

if nota >= 9.0:
    conceito = "A - Excelente"
elif nota >= 7.0:
    conceito = "B - Bom"
elif nota >= 5.0:
    conceito = "C - Regular"
elif nota >= 3.0:
    conceito = "D - Insuficiente"
else:
    conceito = "E - Reprovado"

print(f"Nota: {nota:.1f} -> Conceito: {conceito}")
```

Saída (supondo nota = 8.5):

Nota: 8.5 -> Conceito: B - Bom

⚙️ if-elif-else - Classificação IMC

Código:

```
peso = float(input("Peso (kg): "))
altura = float(input("Altura (m): "))
imc = peso / altura ** 2

if imc < 18.5:
    classificacao = "Abaixo do peso"
elif imc < 25.0:
    classificacao = "Peso normal"
elif imc < 30.0:
    classificacao = "Sobrepeso"
elif imc < 35.0:
    classificacao = "Obesidade Grau I"
elif imc < 40.0:
    classificacao = "Obesidade Grau II"
else:
    classificacao = "Obesidade Grau III"

print(f"IMC: {imc:.2f} - {classificacao}")
```

if-elif-else - Classificação IMC

Saída (supondo peso=70, altura=1.75):

```
## IMC: 22.86 - Peso normal
```

Condicionais Aninhadas

Podemos colocar um **if dentro** de outro **if** para decisões mais complexas.

Código:

```
idade = int(input("Idade: "))
tem_carteira = input("Tem carteira de motorista? (s/n): ")

if idade >= 18:
    if tem_carteira == "s":
        print("Pode dirigir!")
    else:
        print("Maior de idade, mas sem carteira.")
else:
    print("Menor de idade. Não pode dirigir.")
```

Saída (supondo idade=25, tem_carteira="s"):

Pode dirigir!

≡ Condicionais Aninhadas com Operadores Lógicos

Muitas vezes, podemos **simplificar** condicionais aninhadas usando operadores lógicos (**and**, **or**, **not**).

Versão com **and** (equivalente ao slide anterior):

Código:

```
idade = int(input("Idade: "))
tem_carteira = input("Tem carteira? (s/n): ")

if idade >= 18 and tem_carteira == "s":
    print("Pode dirigir!")
elif idade >= 18 and tem_carteira != "s":
    print("Maior de idade, mas sem carteira.")
else:
    print("Menor de idade. Não pode dirigir.")
```

Saída (supondo idade=16):

Menor de idade. Não pode dirigir.

⚡ Operador Ternário

O **operador ternário** permite escrever uma condição simples em **uma única linha**.

Sintaxe:

```
valor = resultado_se_true if condicao else resultado_se_false
```

Código:

```
idade = int(input("Idade: "))
status = "Maior de idade" if idade >= 18 else "Menor de idade"
print(status)

numero = 10
tipo = "par" if numero % 2 == 0 else "impar"
print(f"{numero} é {tipo}")
```

Saída:

Menor de idade

☑ Boas Práticas em Condicionais

Faça:

- Use indentação consistente (4 espaços)
- Simplifique condições complexas com variáveis auxiliares
- Prefira **elif** em vez de vários **if** independentes
- Teste valores extremos (0, negativos, limites)

EVITE:

- Comparar booleanos com `== True` ou `== False`

☑ Boas Práticas em Condicionais

Exemplo:

Código:

```
# RUIM
ativo = True
if ativo == True:
    print("Ativo")

# BOM
if ativo:
    print("Ativo")
```

Saída:

```
## Ativo
```

Exercício 1: Condicionais

1a) Classificador de triângulos

Leia três lados de um triângulo e determine se ele é:

- **Equilátero** (3 lados iguais)
- **Isósceles** (2 lados iguais)
- **Escaleno** (3 lados diferentes)
- Antes, verifique se os valores formam um triângulo válido!

1b) Calculadora de desconto

Uma loja oferece desconto baseado no valor da compra:

- Até R\$ 100: sem desconto
- De R\$ 100,01 a R\$ 500: 10% de desconto
- Acima de R\$ 500: 20% de desconto

Leia o valor da compra e exiba o valor com desconto.

 Exercício 1: Condicionais

1c) Sistema de notas com frequência

Leia nota e frequência (%). Determine:

- APROVADO: nota ≥ 7.0 e frequência $\geq 75\%$
- REPROVADO por nota
- REPROVADO por falta
- REPROVADO por nota e falta

ESTRUTURAS DE REPETIÇÃO (LOOPS)

❓ O que são Estruturas de Repetição?

Estruturas de repetição permitem **executar um bloco de código várias vezes**, de forma controlada, até que uma condição seja satisfeita.

Componentes principais:

1. **Inicialização** - preparar a variável de controle
2. **Condição de parada** - quando o loop deve parar
3. **Atualização** - modificar a variável de controle

Analogia:

Como o Spotify reproduz uma playlist: ele **repete** a ação de tocar música **enquanto** houver músicas na lista.

Tipos de Loops em Python

Tipo	Quando usar	Portugol
<code>for</code>	Sabemos quantas vezes repetir	<code>para ... faça</code>
<code>while</code>	Repetir enquanto condição for verdadeira	<code>enquanto ... faça</code>

Tipos de Loops - Exemplos

Use **for** quando quiser:

- Percorrer uma lista de alunos
- Somar 10 números
- Reproduzir uma playlist

Use **while** quando quiser:

- Validar senha até acertar
- Repetir um menu até sair
- Simular scroll infinito

🔄 Loop **for** - Básico

O **for** percorre uma **sequência** de valores usando **range()**.

Sintaxe:

```
for variavel in range(inicio, fim, passo):  
    # bloco repetido
```

Exemplo: Contar de 1 a 5

Código:

```
for i in range(1, 6):  
    print(i, end=" ")  
print()
```

Saída:

```
## 1 2 3 4 5
```

Atenção: `range(1, 6)` gera os números 1, 2, 3, 4, 5 (o 6 não é incluído).

🔄 Loop **for** - Exemplo Didático

Quando já temos uma sequência pronta, o **for** percorre **item por item**.

Código:

```
itens = ["Arroz", "Feijão", "Leite"]  
  
for item in itens:  
    print(f"Comprar: {item}")
```

Saída:

```
## Comprar: Arroz  
## Comprar: Feijão  
## Comprar: Leite
```

🔄 Entendendo o `range()` - 1 e 2 argumentos

A função `range()` pode receber 1, 2 ou 3 argumentos:

Código:

```
# range(fim) - começa do 0
for i in range(5):
    print(i, end=" ")
print()

# range(inicio, fim) - define onde começar
for i in range(2, 7):
    print(i, end=" ")
print()
```

Saída:

```
## 0 1 2 3 4
```

```
## 2 3 4 5 6
```

🔄 Entendendo o `range()` - 3 argumentos

Código:

```
# range(inicio, fim, passo) - define o incremento
for i in range(0, 20, 5):
    print(i, end=" ")
print()

# Contagem regressiva (passo negativo)
for i in range(10, 0, -2):
    print(i, end=" ")
print()
```

Saída:

```
## 0 5 10 15
```

```
## 10 8 6 4 2
```

🔄 Loop **for** - Exemplo: Tabuada

Código:

```
numero = int(input("Qual tabuada deseja ver? "))  
  
print(f"\n--- Tabuada do {numero} ---")  
for i in range(1, 11):  
    resultado = numero * i  
    print(f"{numero} x {i:2d} = {resultado:3d}")
```

Loop **for** - Exemplo: Tabuada

Saída (supondo numero=7):

```
##  
## --- Tabuada do 7 ---  
  
## 7 x 1 = 7  
## 7 x 2 = 14  
## 7 x 3 = 21  
## 7 x 4 = 28  
## 7 x 5 = 35  
## 7 x 6 = 42  
## 7 x 7 = 49  
## 7 x 8 = 56  
## 7 x 9 = 63  
## 7 x 10 = 70
```

🔄 Loop **for** - Exemplo: Somatório

Exemplo: Somar os N primeiros números naturais

Código:

```
n = int(input("Somar até qual número? "))

soma = 0
for i in range(1, n + 1):
    soma += i
    print(f"Somando {i}: total parcial = {soma}")

print(f"\nSoma de 1 até {n} = {soma}")
print(f"Verificação pela fórmula: {n * (n + 1) // 2}")
```

Loop **for** - Exemplo: Somatório

Saída (supondo $n=5$):

```
## Somando 1: total parcial = 1
## Somando 2: total parcial = 3
## Somando 3: total parcial = 6
## Somando 4: total parcial = 10
## Somando 5: total parcial = 15
```

```
##
## Soma de 1 até 5 = 15
```

```
## Verificação pela fórmula: 15
```

🔄 Loop **for** - Percorrendo Strings

O **for** também pode percorrer **strings**, caractere por caractere:

Código:

```
nome = input("Digite seu nome: ")

print(f"Letras de '{nome}':")
for letra in nome:
    print(f"    -> {letra}")

# Contar vogais
vogais = "aeiouAEIOU"
contagem = 0
for letra in nome:
    if letra in vogais:
        contagem += 1

print(f"'{nome}' tem {contagem} vogal(is).")
```


Loop **for** - Percorrendo Strings

Saída (supondo nome="Python"):

```
## Letras de 'Python':
```

```
##    -> P
```

```
##    -> y
```

```
##    -> t
```

```
##    -> h
```

```
##    -> o
```

```
##    -> n
```

```
## 'Python' tem 1 vogal(is).
```

💡 Desafio Didático: **for**

1. Imprima os números de **1** a **10** e, ao lado, o quadrado de cada um.
2. Leia uma palavra e conte quantas letras **a** ela possui usando **for**.

Desafio extra:

- Mostre também quantas vogais existem no total.

🔄 Loop **while** - Básico

O **while** repete **enquanto** a condição for verdadeira.

Sintaxe:

```
while condicao:  
    # bloco repetido  
    # atualizar variável de controle!
```

Diferente do **for**, no **while** **você** é responsável por atualizar a variável de controle. Se esquecer, o loop será infinito!

Loop **while** - Exemplo Didático

O **while** continua repetindo **enquanto** a meta ainda não foi alcançada.

Código:

```
meta = 20
saldo = 0

while saldo < meta:
    saldo += 5
    print(f"Saldo atual: R$ {saldo}")

print("Meta alcançada!")
```

Saída:

```
## Saldo atual: R$ 5
## Saldo atual: R$ 10
## Saldo atual: R$ 15
## Saldo atual: R$ 20

## Meta alcançada!
```

while - Exemplo: Contagem regressiva

Código:

```
contagem = int(input("Contagem regressiva a partir de: "))  
  
while contagem > 0:  
    print(f"{contagem}...")  
    contagem -= 1  
  
print("LANÇAMENTO!")
```

while - Exemplo: Contagem regressiva

Saída (supondo contagem=5):

```
## 5...
```

```
## 4...
```

```
## 3...
```

```
## 2...
```

```
## 1...
```

```
## LANÇAMENTO!
```

🔄 Loop **while** - Exemplo: Validação de Senha

Exemplo: Validar senha com 3 tentativas

Código:

```
SENHA_CORRETA = "python123"
tentativas = 3

while tentativas > 0:
    senha = input("Digite a senha: ")
    if senha == SENHA_CORRETA:
        print("Acesso liberado!")
        break
    else:
        tentativas -= 1
        print(f"Senha incorreta! {tentativas} tentativa(s) restante(s).")

if tentativas == 0:
    print("Conta bloqueada!")
```

Loop **while** - Exemplo: Validação de Senha

Saída (simulando erros e acerto):

```
## Digite a senha: 1234
## Senha incorreta! 2 tentativa(s) restante(s).
## Digite a senha: abc
## Senha incorreta! 1 tentativa(s) restante(s).
## Digite a senha: python123
## Acesso liberado!
```


🔄 Loop **while** - Exemplo: Fatorial

Cálculo do fatorial usando **while**:

Código:

```
n = int(input("Calcular fatorial de: "))

fatorial = 1
i = 1

while i <= n:
    fatorial *= i
    print(f"{i}! parcial = {fatorial}")
    i += 1

print(f"\n{n}! = {fatorial}")
```

Loop **while** - Exemplo: Fatorial

Saída (supondo $n=5$):

```
## 1! parcial = 1
## 2! parcial = 2
## 3! parcial = 6
## 4! parcial = 24
## 5! parcial = 120
```

```
##
## 5! = 120
```

Comparando: Fatorial com **for**

O mesmo cálculo usando **for**:

Código:

```
n = int(input("Calcular fatorial de: "))

fatorial = 1
for i in range(1, n + 1):
    fatorial *= i

print(f"{n}! = {fatorial}")
```

Comparando: Fatorial com **for**

Saída (supondo $n=5$):

5! = 120

Neste caso, o **for** é **mais simples** porque sabemos exatamente quantas vezes repetir (de 1 até n).

💡 Desafio Didático: `while`

Faça um programa que:

- comece com `saldo = 0`
- some `3` ao saldo enquanto ele for menor que `15`
- mostre o valor do saldo a cada repetição

Desafio extra:

- ao final, informe quantas repetições foram necessárias

❓ Quando usar **for** vs **while**?

Critério	for	while
Número de iterações	Conhecido	Desconhecido
Sequências	Percorre listas, strings, ranges	Repete até a condição mudar
Controle	Automático pelo range()	Manual (você atualiza)
Risco de loop infinito	Baixo	Alto (se esquecer de atualizar)

Regra prática:

- Sabe quantas vezes? Use **for**
- Não sabe quantas vezes? Use **while**

CONTROLE DE FLUXO EM LOOPS

🔲 Comando **break**

O **break** interrompe imediatamente o loop, saindo completamente dele.

Código:

```
# Encontrar o primeiro número divisível por 7 entre 50 e 100
for num in range(50, 101):
    if num % 7 == 0:
        print(f"Primeiro divisível por 7: {num}")
        break
```

Saída:

```
## Primeiro divisível por 7: 56
```


🔵 **break** - Exemplo: Menu com **while True**

O **while True** cria um loop que só para com **break**.

Código:

```
while True:
    opcao = input("Menu: [1]Jogar [2]Config [0]Sair -> ")
    if opcao == "0":
        print("Saindo...")
        break
    print(f"Você escolheu a opção {opcao}")
```

Esse padrão é muito comum para **menus interativos** e **validação de entrada**.

▶▶ Comando `continue`

O `continue` pula a iteração atual e vai para a próxima.

Exemplo: Imprimir apenas números ímpares

Código:

```
for i in range(1, 11):  
    if i % 2 == 0:  
        continue # pula os pares  
    print(f"{i} é ímpar")
```

Saída:

```
## 1 é ímpar  
## 3 é ímpar  
## 5 é ímpar  
## 7 é ímpar  
## 9 é ímpar
```

▶▶ **continue** - Exemplo: Filtrar posts

Exemplo: Rede social filtrando posts para o feed

Código:

```
posts = ["Foto da viagem", "SPAM: Compre agora!", "Receita de bolo",  
         "SPAM: Ganhe dinheiro!", "Meu cachorro", "Tutorial Python"]  
  
print("=== SEU FEED ===")  
for post in posts:  
    if post.startswith("SPAM"):  
        continue # ignora posts de spam  
    print(f" - {post}")
```

▶▶ continue - Exemplo: Filtrar posts

Saída:

```
## === SEU FEED ===  
  
## - Foto da viagem  
## - Receita de bolo  
## - Meu cachorro  
## - Tutorial Python
```

– Comando **pass**

O **pass** não faz nada. Serve como placeholder quando a sintaxe exige um bloco.

Código:

```
for i in range(5):  
    if i == 3:  
        pass # TODO: implementar lógica futura  
    else:  
        print(f"Processando {i}")
```

Saída:

```
## Processando 0  
## Processando 1  
## Processando 2  
## Processando 4
```

O **pass** é útil quando estamos **planejando** o código e queremos definir a estrutura antes de implementar.

Loops Aninhados

Também podemos usar **mais de um laço junto** no mesmo problema.

Quando um loop fica **dentro** de outro, chamamos isso de **loops aninhados**.

Exemplo: Tabela de multiplicação

Código:

```
print("Tabela de Multiplicação (1 a 5):")
print("-" * 30)
for i in range(1, 6):
    for j in range(1, 6):
        print(f"{i*j:4d}", end=" ")
    print() # quebra de linha após cada linha
```

Loops Aninhados

Saída:

```
## Tabela de Multiplicação (1 a 5):
```

```
## -----
```

```
##      1      2      3      4      5
##      2      4      6      8     10
##      3      6      9     12     15
##      4      8     12     16     20
##      5     10     15     20     25
```

Loops Aninhados - Exemplo: Triângulo de Asteriscos

Código:

```
altura = int(input("Altura: "))
for i in range(1, altura + 1):
    print("*" * i)
print("Triângulo invertido:")
for i in range(altura, 0, -1):
    print("*" * i)
```

Saída:

```
## *
## **
## ***
## ****
## *****
```

Triângulo invertido:

```
## *****
## ****
## ***
## **
## *
```


Contadores e Acumuladores

Contador: variável que conta quantas vezes algo acontece.

Acumulador: variável que soma (acumula) valores.

Código:

```
# Ler 5 notas e calcular média
total = 0    # acumulador
aprovados = 0    # contador

for i in range(1, 6):
    nota = float(input(f"Nota {i}: "))
    total += nota    # acumula
    if nota >= 7.0:
        aprovados += 1    # conta

media = total / 5
print(f"\nMédia da turma: {media:.2f}")
print(f"Aprovados: {aprovados} de 5")
```

Contadores e Acumuladores

Saída (simulando notas):

```
## Nota 1: 8.5
```

```
## Nota 2: 6.0
```

```
## Nota 3: 9.0
```

```
## Nota 4: 4.5
```

```
## Nota 5: 7.5
```

```
##
```

```
## Média da turma: 7.10
```

```
## Aprovados: 3 de 5
```

! O que é um Loop Infinito?

Um **loop infinito** é um laço que **nunca termina** sozinho.

Isso acontece quando:

- a condição do **while** continua sempre verdadeira
- a variável de controle não é atualizada
- a lógica de parada foi escrita de forma incorreta

Em geral, o programa fica repetindo a mesma ação até ser interrompido manualmente.

⚠ Cuidado: Loop Infinito!

Um loop infinito acontece quando a **condição nunca se torna falsa**.

```
# PERIGO - Loop infinito!  
contador = 1  
while contador <= 10:  
    print(contador)  
    # Esqueceu de atualizar o contador!  
    # contador += 1  <-- esta linha está faltando!
```

⚠ Como evitar Loops Infinitos

Como evitar:

- Sempre atualize a variável de controle
- Garanta que a condição **convergir** para falsa
- Em caso de dúvida, adicione um limite máximo

Dica de debugging:

```
# Adicione prints temporarios para depurar
for i in range(10):
    print(f"DEBUG: i = {i}") # remova depois
```

✓ Boas Práticas em Loops

Faça:

- Inicialize variáveis **antes** do loop
- Garanta que o loop **termine**
- Use nomes significativos (**i**, **j**, **k** para contadores simples)
- Teste com valores extremos (0, 1, valores grandes)

EVITE:

- Loops infinitos acidentais
- Modificar o contador dentro do corpo do **for**
- Lógica complexa demais dentro de um único loop
- Aninhar muitos loops (máximo 2-3 níveis)

APLICACOES PRATICAS INTEGRADAS

Aplicação: Caixa Registradora - Código

Código:

```
total = 0
itens = 0

while True:
    preco = input("Preço do item (ou 'fim' para encerrar): ")
    if preco.lower() == "fim":
        break
    preco = float(preco)
    if preco <= 0:
        print("Preço inválido! Tente novamente.")
        continue
    total += preco
    itens += 1
    print(f"  Item {itens}: R$ {preco:.2f} | Subtotal: R$ {total:.2f}")

print(f"\n{'='*30}")
print(f"Total: {itens} item(ns)")
print(f"Valor total: R$ {total:.2f}")
```

Usa **while True** + **break**, **continue** para preço inválido, e contadores/acumuladores.

Caixa Registradora - Saída

Saída (simulando compras):

```
## Preço do item (ou 'fim' para encerrar): 15.9
##   Item 1: R$ 15.90 | Subtotal: R$ 15.90
## Preço do item (ou 'fim' para encerrar): 32.5
##   Item 2: R$ 32.50 | Subtotal: R$ 48.40
## Preço do item (ou 'fim' para encerrar): -5.0
## Preço inválido! Tente novamente.
## Preço do item (ou 'fim' para encerrar): 8.75
##   Item 3: R$ 8.75 | Subtotal: R$ 57.15
## Preço do item (ou 'fim' para encerrar): fim
```

```
##
## =====
```

```
## Total: 3 item(ns)
```

```
## Valor total: R$ 57.15
```

➤ Aplicação: Jogo de Adivinhação - Código

Código:

```
import random

secreto = random.randint(1, 100)
tentativas = 0

print("Pensei num número de 1 a 100. Tente adivinhar!")

while True:
    palpite = int(input("Seu palpite: "))
    tentativas += 1

    if palpite < secreto:
        print("MAIOR! Tente um número mais alto.")
    elif palpite > secreto:
        print("MENOR! Tente um número mais baixo.")
    else:
        print(f"PARABÉNS! Acertou em {tentativas} tentativa(s)!")
        break
```

➤ Jogo de Adivinhação - Classificação Final

Depois do loop, podemos classificar o desempenho do jogador:

```
if tentativas <= 3:  
    print("MESTRE!")  
elif tentativas <= 7:  
    print("Muito bom!")  
else:  
    print("Continue praticando!")
```

Jogo de Adivinhação - Saída

Saída (simulando):

```
## Pensei num número de 1 a 100. Tente adivinhar! (secreto=82)
```

```
## Seu palpite: 50
```

```
## MAIOR! Tente um número mais alto.
```

```
## Seu palpite: 75
```

```
## MAIOR! Tente um número mais alto.
```

```
## Seu palpite: 60
```

```
## MAIOR! Tente um número mais alto.
```

```
## Seu palpite: 65
```

```
## MAIOR! Tente um número mais alto.
```

```
## Seu palpite: 82
```

```
## PARABÉNS! Acertou em 5 tentativa(s)!
```

```
## Muito bom!
```

≡ Aplicação: Números Primos - Código

Código:

```
numero = int(input("Verificar se é primo: "))

if numero < 2:
    print(f"{numero} NÃO é primo.")
else:
    e_primo = True
    for i in range(2, int(numero ** 0.5) + 1):
        if numero % i == 0:
            e_primo = False
            break

    if e_primo:
        print(f"{numero} É primo!")
    else:
        print(f"{numero} NÃO é primo (divisível por {i}).")
```

Combina **if-else**, **for**, **break** e até operação matemática (**** 0.5** = raiz quadrada).

≡ Números Primos - Saída

Saída (testando vários números):

```
## 2 É primo!  
## 7 É primo!  
## 15 NÃO é primo (divisível por 3).  
## 23 É primo!  
## 100 NÃO é primo (divisível por 2).
```

Aplicação: Sequência de Fibonacci

Código:

```
n = int(input("Quantos termos de Fibonacci? "))  
  
a, b = 0, 1  
print(f"Sequência de Fibonacci ({n} termos):")  
  
for i in range(n):  
    print(a, end=" ")  
    a, b = b, a + b  
  
print()
```

Fibonacci - Saída

Saída (supondo $n=10$):

```
## Sequência de Fibonacci (10 termos):
```

```
## 0 1 1 2 3 5 8 13 21 34
```

A sequência de Fibonacci aparece na natureza (espirais de girassóis, conchas) e em algoritmos computacionais.

 Exercício 2: Loops

2a) **Números pares** - Imprima todos os números pares de 1 a 50 usando **for**.

2b) **Somatório com while** - Leia números do usuário até ele digitar 0. Ao final, exiba a soma, a quantidade de números e a média.

2c) **Tabuada completa** - Use loops aninhados para gerar a tabuada do 1 ao 10.

2d) **Potência sem operador** - Calcule **base** ****** **expoente** usando apenas multiplicações em um loop (sem usar ******).

Exercício 3: Desafio Integrado

Sistema de Pedidos para Lanchonete

Desenvolva um sistema que:

1. Permita que o cliente faça vários pedidos
2. Processe uma lista de itens do cardápio
3. Pare de aceitar pedidos ao atingir o limite de **5 itens**
4. Ignore itens que estão **em falta**
5. O cliente pode cancelar digitando **"cancelar"**
6. Ao finalizar, mostre: lista de itens, valor total e itens em falta

Exercício 3: Cardápio

Cardápio sugerido:

Item	Preço	Disponível
Hambúrguer	R\$ 25.00	Sim
Pizza	R\$ 35.00	Sim
Suco	R\$ 8.00	Não
Batata Frita	R\$ 15.00	Sim
Sorvete	R\$ 12.00	Não

Use **while True** + **break**, **continue** para itens em falta, e contadores/acumuladores para o total.

☞ Lista de Exercícios para os Alunos

1. Leia um número e informe se ele é positivo, negativo ou zero.
2. Leia duas notas e diga se o aluno foi aprovado, ficou em recuperação ou foi reprovado.
3. Leia a idade de uma pessoa e classifique como criança, adolescente, adulto ou idoso.
4. Leia três valores e mostre qual é o maior deles.

☞ Lista de Exercícios para os Alunos

1. Imprima todos os números pares de 1 a 30 usando **for**.
2. Leia números até o usuário digitar **0** e mostre soma, quantidade e média.
3. Gere a tabuada completa do 1 ao 10 com loops aninhados.
4. Faça um programa que desenhe um retângulo de ***** com **n** linhas e **m** colunas.

Resumo: Estruturas Condicionais

- `if` - executa se condição for verdadeira
- `if-else` - dois caminhos possíveis
- `if-elif-else` - múltiplos caminhos
- Condicionais aninhadas e operadores lógicos
- Operador ternário para condições simples

Resumo: Estruturas de Repetição

- **for** - quando sabemos quantas vezes repetir
- **while** - quando não sabemos quantas vezes
- **break** - interrompe o loop
- **continue** - pula para a próxima iteração
- **pass** - placeholder (não faz nada)
- Loops aninhados, contadores e acumuladores

Referências Bibliográficas

- FORBELLONE, A. L. V.; EBERSPACHER, H. F. **Lógica de Programação**: A Construção de Algoritmos e Estruturas de Dados. 4. ed. São Paulo: Pearson, 2016.
- MANZANO, J. A. N. G.; OLIVEIRA, J. F. **Algoritmos**: Lógica para Desenvolvimento de Programação de Computadores. 28. ed. São Paulo: Saraiva, 2020.

Referências Bibliográficas

- MENEZES, N. N. C. **Introdução a Programação com Python**: Algoritmos e Lógica de Programação para Iniciantes. 4. ed. Rio de Janeiro: Novatec, 2024.
- Documentação Oficial Python: docs.python.org/3

Obrigado!

Prof. Dr. José Jairo de Santana e Silva

Universidade Positivo - 2026/1